Reinforcement-Learned Knowledge–Parameter Topology for Continual Language Model Adaptation

Abstract

Large Language Models (LLMs) have achieved remarkable capabilities through large-scale pre-training and parameter scaling. However, current LLMs remain fundamentally static after deployment. Although an LLM can process newly encountered information through its context window, retrieval systems, or external memory, newly acquired knowledge is generally not incorporated into its internal parameters without additional optimization.

This limitation creates a fundamental gap between current LLMs and biological learning systems: humans continuously incorporate newly acquired knowledge into existing internal representations without globally retraining the entire brain.

We propose Reinforcement-Learned Knowledge–Parameter Topology (RL-KPT), a conceptual architecture for continual parameter-level learning in language models. The central hypothesis is that semantic knowledge can be organized into a structured knowledge space, while the parameters of a sufficiently trained neural language model may exhibit corresponding functional subspaces. Instead of attempting to interpret every individual parameter, RL-KPT introduces reinforcement learning as a meta-level controller that learns a mapping from knowledge states and model activations to parameter-update regions.

The proposed framework explicitly separates three problems: (1) defining a semantic knowledge coordinate system, (2) learning the mapping between knowledge states and parameter regions, and (3) dynamically controlling localized parameter plasticity when new knowledge is encountered. The reinforcement-learning policy produces a parameter-update mask, which determines which parameter regions are allowed to change. The reward function jointly considers new-knowledge acquisition, retention of previously learned knowledge, interference with unrelated knowledge, and update cost.

Formally, the proposed adaptation mechanism is

$$M_t = \pi_\phi(S_t, K_t),$$

followed by

$$\theta_{t+1} = \theta_t + M_t \odot \Delta\theta_t.$$

Here, M_t is a learned parameter-update mask, S_t represents the current model state, and K_t represents the semantic knowledge state.

RL-KPT differs from retrieval augmentation, conventional fine-tuning, parameter-efficient fine-tuning, mixture-of-experts routing, and existing model-editing approaches by treating the knowledge-to-parameter mapping itself as a learnable object. The framework therefore provides a possible path toward LLMs with localized parameter plasticity and continual internal knowledge acquisition.

This paper presents the theoretical formulation, architectural design, training procedure, research hypotheses, and experimental program required to test this hypothesis.

1 Introduction

Large Language Models have become increasingly capable of language understanding, reasoning, coding, knowledge retrieval, and multimodal interaction. Their capabilities are primarily obtained through large-scale pre-training, architectural improvements, and parameter scaling.

Despite these advances, an important structural limitation remains.

Once a conventional LLM has completed pre-training and deployment begins, its parameters are generally treated as fixed:

$$\theta_{t+1} \approx \theta_t.$$

When the model encounters new information, the information may be temporarily available through the context window, retrieved from an external database, or stored in an external memory system. However, the model normally does not autonomously modify its own internal parameters in a localized and controlled manner.

To permanently incorporate new information into the model, conventional approaches generally require fine-tuning, continual pre-training, model editing, or parameter-efficient adaptation.

This creates a fundamental question:

Can an LLM learn not only knowledge, but also where different types of knowledge should be represented inside its parameter space?

If such a relationship can be learned, a second question follows:

Can the model use this learned relationship to selectively modify only the parameter regions relevant to newly acquired knowledge?

RL-KPT is based on the hypothesis that the answer may be yes.

2 Core Hypothesis

The central hypothesis of RL-KPT is:

A sufficiently trained language model contains structured relationships between semantic knowledge states and subsets of its internal parameters, and reinforcement learning can be used to discover and control these relationships for continual localized adaptation.

This hypothesis does not require every fact to correspond to a single isolated parameter.

Instead, a knowledge concept may correspond to a distributed parameter subspace.

For example,

$$P_{\text{physics}}$$

may overlap with

$$P_{\text{mathematics}}$$

and

$$P_{\text{engineering}}.$$

Consequently, complex knowledge can be represented through combinations of parameter regions.

For example:

$$P_{\text{robotics}} \approx P_{\text{physics}} \cap P_{\text{mathematics}} \cap P_{\text{mechanics}} \cap P_{\text{control}}.$$

Therefore, RL-KPT does not require a one-to-one relationship between concepts and parameter groups.

3 Knowledge Space as an Explicit Coordinate System

Human knowledge already possesses relatively stable high-level semantic organization.

We define a knowledge space:

$$\mathcal{K} = \{K_1, K_2, \ldots, K_m\}.$$

Possible basis domains include:

- mathematics,
- physics,
- chemistry,
- biology,
- medicine,
- engineering,
- computer science,
- humanities,
- social sciences,
- business,
- everyday knowledge.

The framework does not require the reinforcement-learning system to rediscover these semantic categories.

Instead, these categories provide an explicit coordinate system.

A new knowledge item x is encoded into a knowledge-state vector:

$$z_x = f_K(x).$$

For example,

$$z_x = [0.05, 0.82, 0.71, 0.63, \ldots]$$

could indicate that the knowledge is strongly associated with physics, engineering, and control.

This distinction is important.

RL-KPT does not ask reinforcement learning to discover what physics is.

It asks reinforcement learning to learn:

$$\boxed{\text{Knowledge Space} \rightarrow \text{Parameter Space}}$$

4 Parameter Topology

Let the model parameters be

$$\theta = \{\theta_1, \theta_2, \ldots, \theta_N\}.$$

Instead of treating these parameters as a completely undifferentiated optimization space, RL-KPT assumes that the parameter space can be represented as a collection of functional regions:

$$\mathcal{P} = \{P_1, P_2, \ldots, P_q\}.$$

A parameter region may be defined at different structural levels:

1. individual neurons,
2. MLP channels,
3. attention heads,
4. layer blocks,

5. low-rank subspaces,
6. structured parameter groups,
7. distributed cross-layer parameter regions.

The exact granularity is not predetermined by the theory.
Instead, the granularity becomes an empirical research question.
The crucial requirement is that the regions are controllable.

5 Knowledge–Parameter Mapping

RL-KPT introduces a mapping:

$$F : \mathcal{K} \times \mathcal{H} \to \mathcal{P},$$

where $\mathcal{H}$ represents the model's internal activation state.

The mapping can be implemented through a reinforcement-learning policy:

$$M_t = \pi_\phi(S_t, K_t),$$

where:

- S_t is the current model state,
- K_t is the knowledge-space representation,
- π_ϕ is the RL policy,
- M_t is the parameter-update mask.

The mask can be binary:

$$M_i \in \{0, 1\},$$

or continuous:

$$M_i \in [0, 1].$$

A continuous mask allows the controller to regulate update intensity.

6 Reinforcement Learning Formulation

RL-KPT formulates parameter-region selection as a Markov Decision Process.

6.1 State

The state is:

$$S_t = (h_t, z_t, c_t, \theta_t),$$

where:

- h_t: transformer hidden states,
- z_t: knowledge-space representation,
- c_t: contextual information,
- θ_t: current model state.

6.2 Action

The action is a parameter-region selection:

$$A_t = M_t.$$

For example:

$$M_t = [0, 0.1, 0.8, 0.7, 0, \ldots].$$

This means that some regions are frozen, some are weakly updated, and some are strongly updated.

6.3 Reward

The reward should not simply measure immediate task accuracy.

We define:

$$R_t = \lambda_1 R_{\text{learn}} + \lambda_2 R_{\text{retain}} + \lambda_3 R_{\text{transfer}} - \lambda_4 R_{\text{interference}} - \lambda_5 R_{\text{cost}}.$$

Where:

New knowledge acquisition

$$R_{\text{learn}}$$

measures whether the new knowledge has been successfully incorporated.

Knowledge retention

$$R_{\text{retain}}$$

measures whether previously learned knowledge remains intact.

Transfer

$$R_{\text{transfer}}$$

measures whether the learned knowledge improves related tasks.

Interference

$$R_{\text{interference}}$$

penalizes degradation of unrelated knowledge.

Update cost

$$R_{\text{cost}}$$

penalizes unnecessarily large parameter modifications.

The reward therefore encourages:

maximum useful knowledge acquisition with minimum unnecessary parameter change.

7 Local Parameter Plasticity

Conventional fine-tuning applies updates according to the gradient:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L.$$

RL-KPT introduces an additional structural constraint:

$$\boxed{\theta_{t+1} = \theta_t + M_t \odot \Delta\theta_t}$$

where:

$$M_t = \pi_\phi(S_t, K_t).$$

The RL controller therefore does not necessarily replace gradient-based optimization.
Instead, it controls where gradient-based adaptation is allowed to occur.
This distinction is central.
RL determines:

where to learn

while gradient optimization determines:

how to modify the selected parameters.

8 Training Procedure

The proposed training process contains two conceptual stages.

Stage I: Foundation Model Training

A conventional language model is pretrained.
The resulting model provides general language and reasoning capabilities.

Stage II: Knowledge–Parameter Topology Learning

Training examples are associated with semantic knowledge domains.
For each training episode:

1. sample a knowledge instance;
2. encode its semantic knowledge state;
3. observe transformer activation patterns;
4. allow the RL controller to select parameter regions;
5. perform localized adaptation;
6. evaluate new-knowledge acquisition;
7. evaluate preservation of previous knowledge;
8. calculate the reward;
9. update the RL policy.

Over repeated episodes, the policy is expected to learn:

$$\pi_\phi : (K, S) \to M.$$

The learned policy therefore becomes a parameter-routing strategy.

9 Algorithm 1: RL-KPT Training Procedure

Algorithm 1: Reinforcement-Learned Knowledge–Parameter Topology Training

Input: pretrained language model f_θ, knowledge-domain set $\mathcal{K}$, training stream $\mathcal{D}$, RL policy π_ϕ

Initialize: parameter-region representation $\mathcal{P}$, RL policy parameters ϕ

For each training episode t:

1. Sample a knowledge item $x_t \sim \mathcal{D}$.
2. Determine its knowledge-space representation:
$$z_t = f_K(x_t).$$
3. Run the language model and obtain activation state:
$$h_t = f_\theta(x_t).$$
4. Construct RL state:
$$S_t = (h_t, z_t, c_t, \theta_t).$$
5. Generate a parameter-update mask:
$$M_t = \pi_\phi(S_t).$$
6. Compute a localized parameter update:
$$\Delta\theta_t = -\eta \nabla_\theta L(x_t).$$
7. Apply the mask:
$$\theta_{t+1} = \theta_t + M_t \odot \Delta\theta_t.$$
8. Evaluate:
   - new knowledge acquisition,
   - old knowledge retention,
   - related-task transfer,
   - unrelated-task interference,
   - update cost.
9. Compute:
$$R_t = \lambda_1 R_{\text{learn}} + \lambda_2 R_{\text{retain}} + \lambda_3 R_{\text{transfer}} - \lambda_4 R_{\text{interference}} - \lambda_5 R_{\text{cost}}.$$
10. Update the RL policy:
$$\phi \leftarrow \text{RLUpdate}(\phi, S_t, M_t, R_t).$$
11. Continue until the knowledge–parameter routing policy converges.

Output: pretrained model with learned knowledge–parameter routing policy π_ϕ.

10 Online Continual Learning

After training, the model can enter an interactive learning stage.

Suppose a human provides new information:
Error code E27 in the X-series servo drive indicates an encoder communication failure.
The model performs:
$$x_{\text{new}} \to z_{\text{new}} \to h_{\text{new}} \to \pi_\phi \to M_{\text{new}}.$$
The resulting mask may activate regions associated with:
$$\{\text{engineering}, \text{mechanics}, \text{electrical control}, \text{servo systems}, \text{diagnosis}\}.$$
Only the corresponding parameter regions are updated.
Thus:
$$\theta_{t+1} = \theta_t + M_{\text{new}} \odot \Delta\theta.$$
The new knowledge is therefore not merely retrieved from an external memory.
It becomes part of the model's evolving internal parameter state.

11 Knowledge Composition

The framework does not require each knowledge domain to have an isolated parameter region.

Instead, knowledge domains may overlap.

For example:

$$P_{\text{robotics}} \approx P_{\text{mechanics}} \cup P_{\text{control}} \cup P_{\text{physics}} \cup P_{\text{mathematics}}.$$

This allows the architecture to represent interdisciplinary knowledge.

A new knowledge item may therefore activate:

$$M_{\text{new}} = M_{\text{physics}} + M_{\text{mechanics}} + M_{\text{control}}.$$

This compositional property is essential for generalization.

12 Why Reinforcement Learning?

The role of reinforcement learning is not to replace backpropagation.

Instead, RL addresses a different optimization problem:

$$\boxed{\text{Which parameter regions should be modified?}}$$

Gradient descent answers:

$$\boxed{\text{How should selected parameters be modified?}}$$

This separation allows the model to learn a meta-level policy over parameter plasticity.

The RL controller therefore operates one level above conventional parameter optimization.

13 Relation to Mixture-of-Experts

Mixture-of-Experts architectures route computation to different experts.

Conceptually:

$$x \rightarrow \text{Router} \rightarrow \text{Experts}.$$

RL-KPT instead proposes:

$$x \rightarrow \text{Knowledge State} \rightarrow \text{RL Router} \rightarrow \text{Parameter Update Region}.$$

The distinction is therefore:

MoE routes computation.

RL-KPT routes parameter plasticity.

The two mechanisms could potentially coexist.

14 Relation to Parameter-Efficient Fine-Tuning

Methods such as LoRA reduce the number of trainable parameters by introducing low-rank adaptation matrices.

RL-KPT addresses a different problem.

Instead of asking:

Which low-rank adapter should be trained?

it asks:

Which internal parameter regions should be allowed to change for this knowledge state?

LoRA could therefore potentially serve as the local update mechanism inside an RL-KPT region.

15 Relation to Knowledge Editing

Knowledge editing methods attempt to modify specific facts in pretrained language models.

RL-KPT shares the goal of localized knowledge modification but differs in scope.

Knowledge editing generally asks:

How can a particular fact be edited?

RL-KPT asks:

How can a model learn a reusable policy for routing future knowledge to parameter regions?

Therefore, RL-KPT is intended as a continual adaptation architecture, rather than a one-shot editing algorithm.

16 Relation to Continual Learning

Continual-learning research focuses primarily on preventing catastrophic forgetting while learning sequential tasks.

RL-KPT adds a structural hypothesis:

Catastrophic interference may be reduced if the model learns a reusable mapping between knowledge states and localized parameter plasticity.

Thus, the problem changes from:

learn new task without forgetting old task

to:

learn where new knowledge should be incorporated.

17 Research Hypotheses

The framework leads to five experimentally testable hypotheses.

H1: Knowledge–Parameter Topology

After RL training, semantically related knowledge should produce statistically distinguishable parameter-region activation patterns.

H2: Routing Stability

The learned knowledge-to-parameter mapping should remain stable across different examples belonging to the same knowledge domain.

H3: Localized Learning

Localized parameter updates should achieve comparable new-knowledge acquisition with substantially fewer modified parameters than full fine-tuning.

H4: Reduced Interference

Localized updates should reduce degradation of unrelated knowledge.

H5: Continual Accumulation

Sequentially learned knowledge should accumulate over multiple interaction cycles without requiring complete retraining.

18 Experimental Program

A small-scale experimental program should be sufficient to test the core hypothesis.

Experiment A: Parameter Topology Discovery

Use a small transformer.

Train on several clearly separated domains:

- mathematics,
- physics,
- biology,
- history.

Measure parameter activation patterns.

Test whether the domains form distinguishable regions.

Experiment B: RL Parameter Routing

Train an RL controller to select parameter masks.

Compare:

1. random masks,
2. gradient-based masks,
3. learned routing,
4. RL-KPT.

Measure:

- new knowledge accuracy,
- parameter modification ratio,
- interference.

Experiment C: Sequential Knowledge Acquisition

Present knowledge sequentially:

$$K_1 \rightarrow K_2 \rightarrow K_3 \rightarrow K_4.$$

After each update, evaluate all previously learned knowledge.

Measure:

$$\text{Forgetting}_t = \text{Accuracy}_{before} - \text{Accuracy}_{after}.$$

Experiment D: Cross-Domain Knowledge

Introduce knowledge requiring multiple domains.

For example:

$$\text{Physics} + \text{Mathematics} + \text{Engineering}.$$

Test whether RL learns compositional parameter routing rather than requiring a completely new isolated module.

19 Potential Advantages

If experimentally validated, RL-KPT could provide:

1. continual parameter-level learning;
2. localized knowledge updates;
3. reduced catastrophic interference;
4. lower adaptation cost;
5. reusable knowledge-to-parameter routing;
6. a more structured interpretation of internal parameter organization.

More importantly, the framework changes the role of the LLM parameter space.

Instead of viewing parameters solely as optimization variables, the parameter space becomes a structured substrate for continual learning.

20 Limitations and Open Problems

Several major questions remain open.

20.1 Does a stable knowledge–parameter topology actually exist?

The framework assumes that sufficiently trained transformers contain sufficiently stable functional regions.

This is an empirical hypothesis rather than an established fact.

20.2 Are knowledge regions truly localized?

Knowledge may be distributed across many layers and components.

RL-KPT therefore does not require strict localization, but requires controllable parameter subspaces.

20.3 Knowledge verification

A model should not automatically internalize incorrect information.

A practical system requires a verification mechanism before permanent parameter modification.

20.4 Scalability

The computational cost of learning an RL policy over extremely large parameter spaces remains an important challenge.

20.5 Security

Autonomous parameter updates introduce risks of malicious knowledge injection and model poisoning.

20.6 Stability

The learned topology itself may evolve as new knowledge accumulates.

Maintaining a stable routing policy while allowing controlled expansion is an important open problem.

21 Broader Implications

The proposed framework suggests a possible transition between two types of language models.

Static LLM

$$\text{Data} \rightarrow \text{Training} \rightarrow \text{Fixed Parameters}$$

Continually Plastic LLM

Experience → Knowledge Representation → Parameter Routing → Local Plasticity → Updated Model

The second architecture is closer to a continually adapting system.

This does not imply human-level intelligence.

However, it introduces an architectural mechanism that is largely absent from conventional frozen LLM deployment:

$$\boxed{\text{Experience} \rightarrow \text{Internal Parameter Change}}$$

22 Conclusion

We proposed Reinforcement-Learned Knowledge–Parameter Topology (RL-KPT), a conceptual framework for continual parameter-level adaptation in large language models.

The key idea is to provide the model with an explicit semantic knowledge coordinate system and use reinforcement learning to learn a mapping from knowledge states and model activations to parameter-update masks.

The resulting adaptation process is:

Knowledge → Knowledge Space → RL Routing → Parameter Region → Local Plasticity.

Unlike conventional retrieval, fine-tuning, or model editing, RL-KPT treats the knowledge-to-parameter mapping itself as a learnable control problem.

The framework does not claim that such a topology has already been demonstrated. Instead, it proposes a falsifiable research hypothesis:

If reinforcement learning can discover stable and reusable knowledge–parameter routing, then LLMs may acquire a form of localized continual parameter plasticity, allowing newly encountered knowledge to be incorporated without globally retraining the model.

Testing this hypothesis may provide a new direction for the development of continually learning language models beyond pure parameter scaling.

References

[1] Vaswani, A., et al. Attention Is All You Need. arXiv:1706.03762, 2017.

[2] Kirkpatrick, J., et al. Overcoming Catastrophic Forgetting in Neural Networks. arXiv:1612.00796, 2016.

[3] Li, Z., and Hoiem, D. Learning without Forgetting. arXiv:1606.09282, 2016.

[4] Rusu, A. A., et al. Progressive Neural Networks. arXiv:1606.04671, 2016.

[5] Mallya, A., and Lazebnik, S. PackNet: Adding Multiple Tasks to a Single Network by Iterative Pruning. arXiv:1711.05769, 2017.

[6] Mallya, A., Davis, D., and Lazebnik, S. Piggyback: Adapting a Single Network to Multiple Tasks by Learning to Mask Weights. arXiv:1801.06519, 2018.

[7] Rosenbaum, C., Klinger, T., and Riemer, M. Routing Networks: Adaptive Selection of Non-linear Functions for Multi-Task Learning. arXiv:1711.01239, 2017.

[8] Amer, M., and Maul, T. Reducing Catastrophic Forgetting in Modular Neural Networks by Dynamic Information Balancing. arXiv:1912.04508, 2019.

[9] Fedus, W., Zoph, B., and Shazeer, N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv:2101.03961, 2021.

[10] Hu, E. J., et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.

[11] De Cao, N., Aziz, W., and Titov, I. Editing Factual Knowledge in Language Models. arXiv:2104.08164, 2021.

[12] Mitchell, E., et al. Fast Model Editing at Scale. arXiv:2110.11309, 2021.

[13] Mitchell, E., et al. Memory-Based Model Editing at Scale. arXiv:2206.06520, 2022.

[14] Meng, K., et al. Locating and Editing Factual Associations in GPT. arXiv:2202.05262, 2022.

[15] Meng, K., et al. Mass-Editing Memory in a Transformer. arXiv:2210.07229, 2022.

[16] Valipour, M., et al. DyLoRA: Parameter Efficient Tuning of Pre-trained Models using Dynamic Search-Free Low-Rank Adaptation. arXiv:2210.07558, 2022.

[17] Ke, Z., et al. Continual Pre-training of Language Models. arXiv:2302.03241, 2023.

[18] Wang, X., et al. Orthogonal Subspace Learning for Language Model Continual Learning. arXiv:2310.14152, 2023.

[19] Wang, S., et al. Knowledge Editing for Large Language Models: A Survey. arXiv:2310.16218, 2023.

[20] Hase, P., et al. Does Localization Inform Editing? Surprising Differences in Causality-Based Localization vs. Knowledge Editing in Language Models. arXiv:2301.04213, 2023.

[21] Yang, Y., et al. Recent Advances of Foundation Language Models-based Continual Learning: A Survey. arXiv:2405.18653, 2024.

[22] Li, H., et al. Theory on Mixture-of-Experts in Continual Learning. arXiv:2406.16437, 2024.

[23] Tong, K., et al. Analytic Subspace Routing: How Recursive Least Squares Works in Continual Learning of Large Language Model. arXiv:2503.13575, 2025.

[24] Chen, X., et al. Learning to Self-Evolve. arXiv:2603.18620, 2026.

[25] Ji, T., et al. Learning What to Remember and What to Internalize in LLM Self-Evolution via Adaptive Memory-Parameter Coordination. arXiv:2608.01234, 2026.

Visualization: RL-KPT Architecture Flow

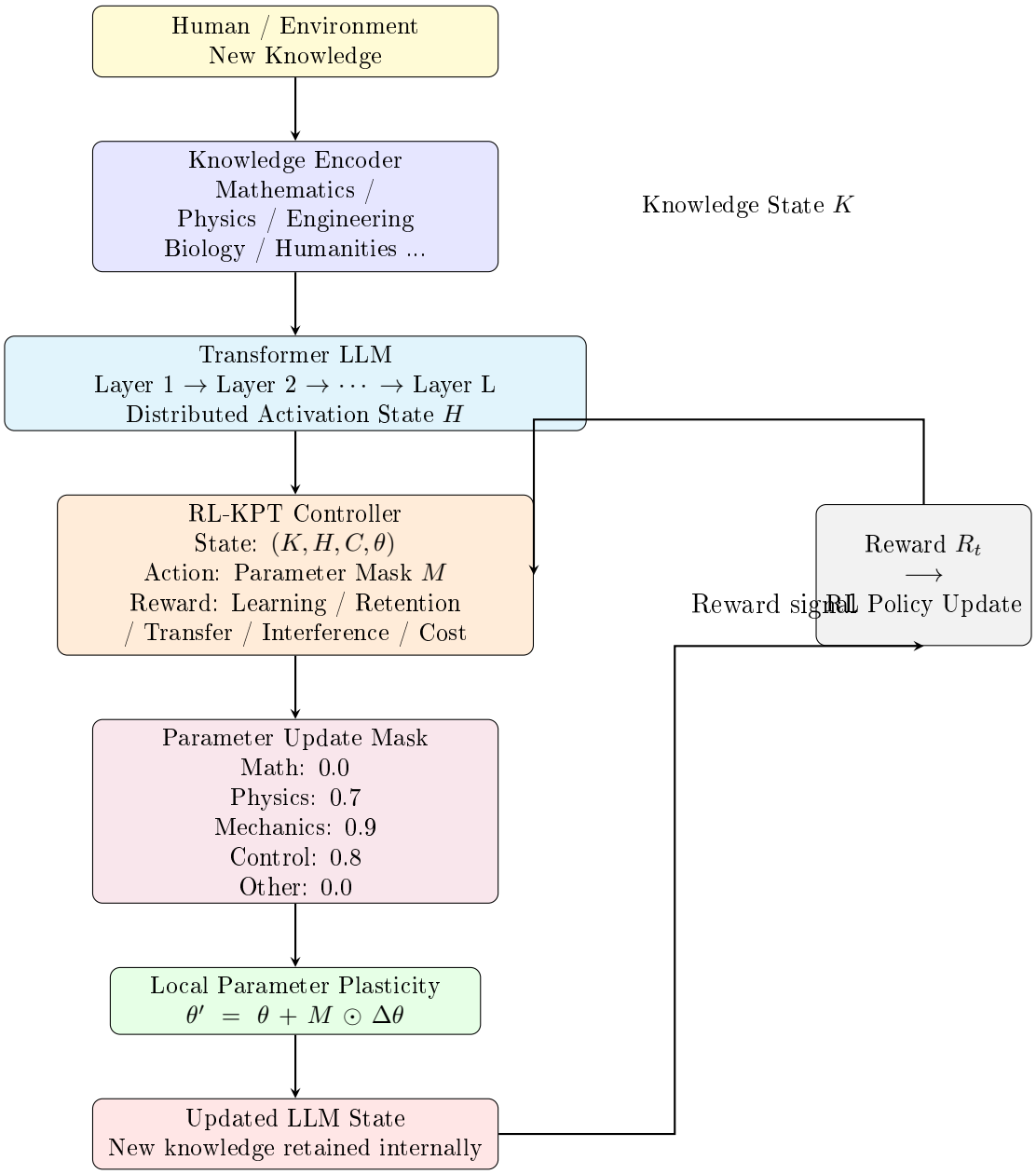

Figure 1: Complete RL-KPT architecture flow. New knowledge is encoded into a knowledge state K, processed by the LLM to obtain activation state H, and fed into the RL-KPT controller. The controller outputs a parameter-update mask M, which is applied to local parameter plasticity. The updated model state is evaluated, and the reward signal is used to update the RL policy.